```
      .oauth2Login(Customizer.withDefaults())
      .logout(logout -> logout.logoutSuccessHandler(                ❶
        oidcLogoutSuccessHandler(clientRegistrationRepository)))
      .build();
  }

  private ServerLogoutSuccessHandler oidcLogoutSuccessHandler(
    ReactiveClientRegistrationRepository clientRegistrationRepository
  ) {
    var oidcLogoutSuccessHandler =
        new OidcClientInitiatedServerLogoutSuccessHandler(
          clientRegistrationRepository);
    oidcLogoutSuccessHandler
      .setPostLogoutRedirectUri("{baseUrl}");                      ❷
    return oidcLogoutSuccessHandler;
  }
}
```

❶ Defines a custom handler for the scenario where a logout operation is completed successfully

❷ After logging out from the OIDC Provider, Keycloak will redirect the user to the application base URL computed dynamically from Spring (locally, it's http://localhost:9000).

NOTE The `ReactiveClientRegistrationRepository` bean is automatically configured by Spring Boot for storing the information about the clients registered with Keycloak, and it's used by Spring Security for authentication/ authorization purposes. In our example, there's only a client: the one we configured earlier in the application.yml file.

I won't ask you to test the logout functionality just yet. The reason will be apparent after we introduce the Angular frontend to the Polar Bookshop system.

The user authentication feature based on OpenID Connect/OAuth2 is now complete, including logout and scalability concerns. If Edge Service used a template engine like Thymeleaf to build the frontend, the work we've done so far would be enough. However, when you're integrating a secured backend application with an SPA like Angular, there are a few more aspects to consider. That will be the focus of the next section.

11.5 Integrating Spring Security with SPAs